

TP 7: Création d'Images Docker Personnalisées

Réalisé par : Bouyahi Mohamed

Exercice 1 : Création d'une Première Image

1. Création un répertoire de projet monProjet

```
root@osboxes:/home/osboxes# mkdir monProjet
```

Création du fichier **index.html** :

2.

Création d'un Dockerfile utilisant NGINX

nano Dockerfile

1. Utiliser NGINX comme une image de base

FROM nginx:latest

2. Copier la page HTML dans où NGINX sert les fichiers

COPY index.html /usr/share/nginx/html/index.html

3. Construire l'image Docker

```

root@osboxes:/home/osboxes/monProjet# docker build -t monprojet .
DEPRECATED: The legacy builder is deprecated and will be removed in a future
release.

          Install the buildx component to build images with BuildKit:
          https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/2 : FROM nginx:latest
--> 177015cbaee5
Step 2/2 : COPY index.html /usr/share/nginx/html/index.html
--> 58e5ed517cdc
Successfully built 58e5ed517cdc
Successfully tagged monprojet:latest

```

```

root@osboxes:/home/osboxes# docker images

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
monprojet	latest	58e5ed517cdc	5 hours ago	152MB
nginx	latest	177015cbaee5	21 hours ago	152MB
ubuntu	latest	c3a134f2ace4	7 weeks ago	78.1MB
hello-world	latest	1b44b5a3e06a	4 months ago	10.1kB

4. Exécuter un conteneur depuis l'image créée

```
docker run -d -p 8080:80 --name web1 monsite:v1
```



A screenshot of a web browser window. The address bar shows 'localhost:8080'. The main content area displays 'Hello, World!' in a bold, black, sans-serif font.

- Créer une image optimisée avec Alpine

```

# Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
# 1. Utiliser NGINX comme une image de base
FROM nginx:latest

# 2. Copier la page HTML dans où NGINX sert les fichiers
COPY index.html /usr/share/nginx/html/index.html

```

Construire l'image optimisée : `docker build -t monprojet:alpine .`

```

Status: Downloaded newer image for nginx:alpine
---> d4918ca78576
Step 2/4 : COPY index.html /usr/share/nginx/html/index.html
---> c37bf44c64a6
Step 3/4 : FROM nginx:latest
---> 177015cbaee5
Step 4/4 : COPY index.html /usr/share/nginx/html/index.html
---> Using cache
---> 0dfc4c5e01a0
Successfully built 0dfc4c5e01a0
Successfully tagged monprojet:alpine
root@osboxes:/home/osboxes/monProjet# docker build --no-cache -t monprojet:alpine .
DEPRECATED: The legacy builder is deprecated and will be removed in a future
release.

          Install the buildx component to build images with BuildKit:
          https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/4 : FROM nginx:alpine
---> d4918ca78576
Step 2/4 : COPY index.html /usr/share/nginx/html/index.html
---> 828b13930f69
Step 3/4 : FROM nginx:latest
---> 177015cbaee5
Step 4/4 : COPY index.html /usr/share/nginx/html/index.html
---> 13154d24f2a1
Successfully built 13154d24f2a1
Successfully tagged monprojet:alpine

```

5. Comparer la taille de l'image originale et la taille de l'image optimisée

```

root@osboxes:/home/osboxes/monProjet# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
43a9e3eb7664   monprojet:alpine  "/docker-entrypoint..." 11 seconds ago Up 10 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp monweb-alpine
d015d52da77d   monprojet:v1     "/docker-entrypoint..." 22 minutes ago Up 22 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp monweb
root@osboxes:/home/osboxes/monProjet# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
<none>        <none>    828b13930f69   3 minutes ago  52.8MB
monprojet     alpine    13154d24f2a1   3 minutes ago  152MB
<none>        <none>    c37bf44c64a6   3 minutes ago  52.8MB
monprojet     v1        0dfc4c5e01a0   41 minutes ago 152MB
monprojet     latest    58e5ed517cdc   6 hours ago    152MB
nginx         latest    177015cbaee5   22 hours ago   152MB
nginx         alpine    d4918ca78576   5 weeks ago    52.8MB
ubuntu        latest    c3a134f2ace4   7 weeks ago    78.1MB
hello-world   latest    1b44b5a3e06a   4 months ago   10.1kB
root@osboxes:/home/osboxes/monProjet#

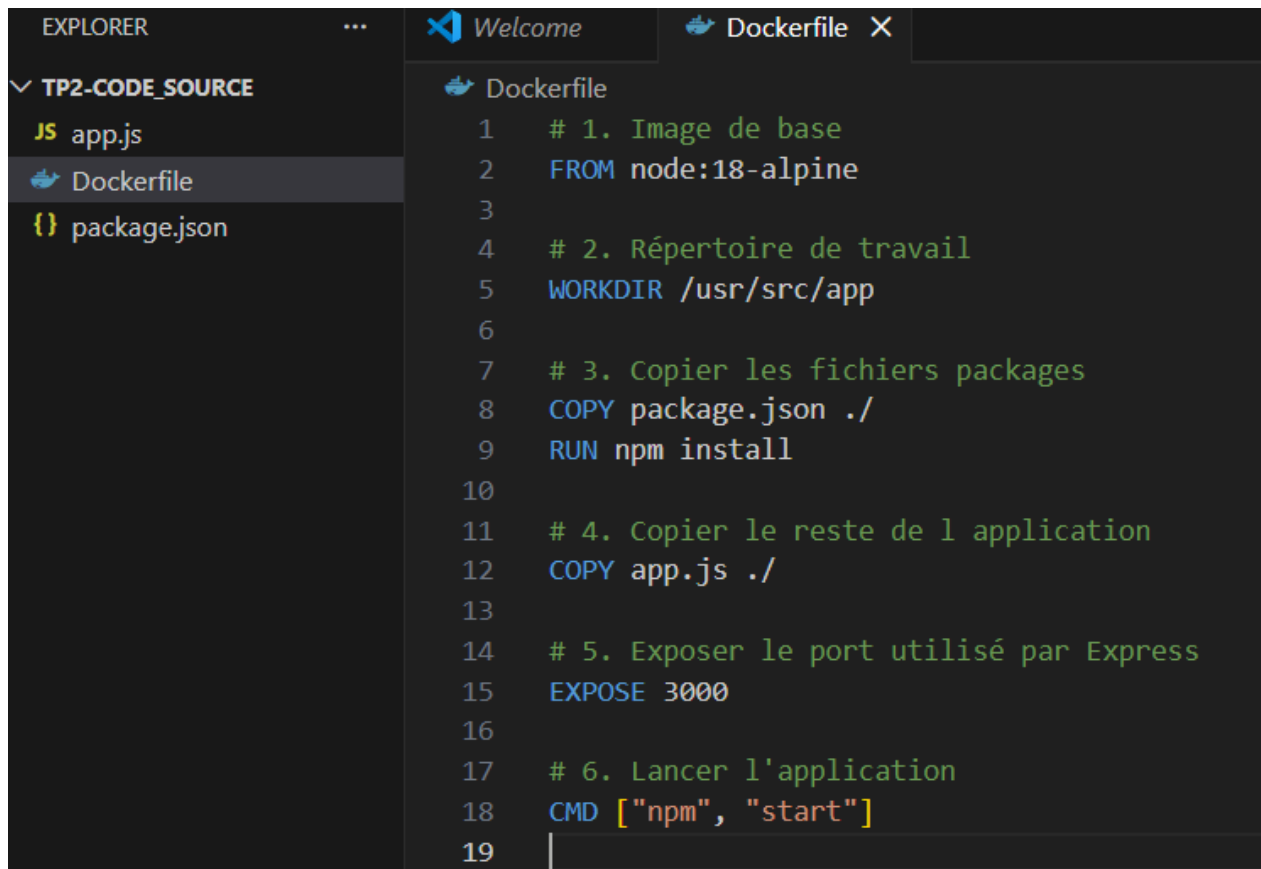
```

Image	Taille
monprojet:v1	152MB
monprojet:alpine	~52MB ✓
nginx:alpine	52MB

Réduction de taille : ~**100MB** (≈ 66% d'économie)

Exercice 2 : Dockeriser une application Node.js (Express)

- Ecrire le Dockerfile



The screenshot shows the Visual Studio Code interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'TP2-CODE_SOURCE' containing 'app.js', 'Dockerfile', and 'package.json'. The 'Dockerfile' is selected and its content is displayed in the code editor. The Dockerfile contains six steps: 1. Base image (node:18-alpine), 2. Working directory (/usr/src/app), 3. Copying package files (package.json), 4. Copying the rest of the application (app.js), 5. Exposing the port (3000), and 6. Running the application (npm start).

```

1  # 1. Image de base
2  FROM node:18-alpine
3
4  # 2. Répertoire de travail
5  WORKDIR /usr/src/app
6
7  # 3. Copier les fichiers packages
8  COPY package.json ./
9  RUN npm install
10
11 # 4. Copier le reste de l'application
12 COPY app.js ./
13
14 # 5. Exposer le port utilisé par Express
15 EXPOSE 3000
16
17 # 6. Lancer l'application
18 CMD ["npm", "start"]
19

```

```

[+] Building 588.9s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                 0.7s
=> => transferring dockerfile: 376B                                0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine   4.7s
=> [internal] load .dockerignore                                    0.1s
=> => transferring context: 2B                                       0.1s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4 561.2s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c2 0.1s
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab2 446B / 446B 0.5s
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96ed 1.26MB / 1.26MB 58.2s
=> => sha256:dd71dde834b5c203d162902e6b8994cb230 40.01MB / 40.01MB 559.2s
=> => sha256:f18232174bc91741fdf3da96d85011092101a 3.64MB / 3.64MB 136.0s
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a 0.5s
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0e 1.7s
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837 0.0s
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab22 0.0s
=> [internal] load build context                                    0.1s
=> => transferring context: 821B                                       0.0s
=> [2/5] WORKDIR /usr/src/app                                       0.2s
=> [3/5] COPY package.json ./                                       0.1s
=> [4/5] RUN npm install                                           20.5s
=> [5/5] COPY app.js ./                                             0.1s
=> exporting to image                                              0.9s
=> => exporting layers                                                0.4s
=> => exporting manifest sha256:5e3fdb8a2be4e89974c0df784619faf50e19 0.0s
=> => exporting config sha256:94ad403636327722cb538aae00195e7e0fef55 0.0s
=> => exporting attestation manifest sha256:fc7a8e834773ba8f0675805e 0.0s
=> => exporting manifest list sha256:638a3d2a1e09438ca8732103a972d43 0.0s
=> => naming to docker.io/library/express-app:latest                0.0s
=> => unpacking to docker.io/library/express-app:latest              0.4s

```

2. Lancer (exécuter) l'image dans un conteneur

```
docker run -p 3000:3000 --name expressnegui-container express-app
```

```

D:\ingenieurie4\Cloud\TPS\TP7-V02-CreationImage\TP2-Code_source> docker run -
p 3000:3000 --name express-container express-app

> express-docker-example@1.0.0 start
> node app.js

Serveur démarré sur http://localhost:3000

```

On exécute l'image, on connecte le port 3000 local au port 3000 dans Docker, on a nommé le container expressnegui-container et on utilise l'image qu'on a construite au préalable.

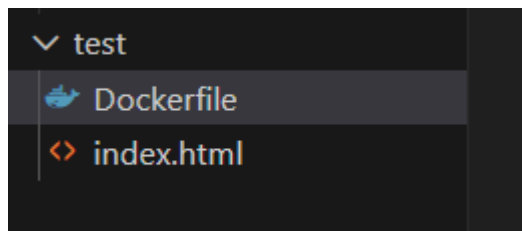
```
← → ↻ ⓘ localhost:3000
Impression élégante ✓
{
  "message": "API Express fonctionne !"
}
```

```
← → ↻ ⓘ localhost:3000/hello
Impression élégante ✓
{
  "message": "Bonjour depuis Docker !"
}
```

Exercice 3 : Comprendre CMD et ENTRYPOINT

● PARTIE A — Analyse de CMD

6.



7.

```
test > Dockerfile
1 FROM nginx:alpine
2
3 # Copier la page HTML
4 COPY index.html /usr/share/nginx/html/
5
6 # Afficher le contenu du répertoire dans le conteneur
7 CMD ["ls", "-l", "/usr/share/nginx/html"]
8
```

8. Construire l'image

```
D:\ingeneurie4\Cloud\TPS\TP7-V02-CreationImage\test>docker build -t test-cmd .
[+] Building 220.7s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 224B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 62B
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:289decab414250121a93c3f1b8316b9c69906de3a4993757c424cb9641
=> => resolve docker.io/library/nginx:alpine@sha256:289decab414250121a93c3f1b8316b9c69906de3a4993757c424cb964169
=> => sha256:136bc6976c2023e3363e66b88167d08019fece3e756c162c58754e3819bf4063 17.26MB / 17.26MB
=> => sha256:fc13532503d72b70e7dd276ae52f2743b14326b83c31935c86d7477c66019dea 1.40kB / 1.40kB
=> => sha256:51c30493937c33bd8b568d8aed09d9596f558d08877b05a5e1855516aba05e1f 403B / 403B
=> => sha256:ad5b65da02cfbd43daa87443b87051f3816a10eb7719938d8cb9a96ee828d471 1.21kB / 1.21kB
=> => sha256:abdece946203a31d986f184559f417a33c3a8936a80153b2f0ffa208af4a0d48 954B / 954B
=> => sha256:5d2cc344426d3d91200b457a771ecfe976de824e165506f5cce5d6b863da1ca9 629B / 629B
=> => sha256:dfad290a5c259f8d1ec1938529f8ef602e335a26680497ad56d38e0727e1a10a 1.86MB / 1.86MB
=> => sha256:014e56e613968f73cce0858124ca5fbc601d7888099969a4eea69f31dcd71a53 3.86MB / 3.86MB
=> => extracting sha256:014e56e613968f73cce0858124ca5fbc601d7888099969a4eea69f31dcd71a53
=> => extracting sha256:dfad290a5c259f8d1ec1938529f8ef602e335a26680497ad56d38e0727e1a10a
=> => extracting sha256:5d2cc344426d3d91200b457a771ecfe976de824e165506f5cce5d6b863da1ca9
=> => extracting sha256:abdece946203a31d986f184559f417a33c3a8936a80153b2f0ffa208af4a0d48
=> => extracting sha256:51c30493937c33bd8b568d8aed09d9596f558d08877b05a5e1855516aba05e1f
=> => extracting sha256:ad5b65da02cfbd43daa87443b87051f3816a10eb7719938d8cb9a96ee828d471
=> => extracting sha256:fc13532503d72b70e7dd276ae52f2743b14326b83c31935c86d7477c66019dea
=> => extracting sha256:136bc6976c2023e3363e66b88167d08019fece3e756c162c58754e3819bf4063
```

9. Exécuter le conteneur

```
D:\ingeneurie4\Cloud\TPS\TP7-V02-CreationImage\test>docker run --name testcmd-container test-cmd
total 8
-rw-r--r-- 1 root root 497 Dec 9 19:41 50x.html
-rwxr-xr-x 1 root root 25 Dec 10 22:40 index.html
```

10. Arrêter et supprimer le conteneur

```
D:\ingeneurie4\Cloud\TPS\TP7-V02-CreationImage\test>docker rm testcmd-contai
ner
testcmd-container
```

11. Relancer le conteneur en remplaçant CMD

```
D:\ingeneurie4\Cloud\TPS\TP7-V02-CreationImage\test>docker run --name testcm
d-container test-cmd ls -a /
.
..
.dockerenv
bin
dev
docker-entrypoint.d
docker-entrypoint.sh
etc
home
lib
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
```